

Module 2: Syntax Analysis

Context-Free Grammars, LL(1), SLR(1), CLR(1), LALR(1) & Parsing Conflicts

Module II: Syntax Analysis – Complete 10-Topic Syllabus Tracker

Topic 1: Introduction to Syntax Analysis & CFG Grammars

Topic 3: Recursive Top-Down Parsers (Recursive Descent & Backtracking)

Topic 5: Design of LL(1) Parser (FIRST & FOLLOW Inductive Rules)

Topic 7: Variants of LR Parsers (LR(0), SLR(1), CLR(1), LALR(1))

Topic 9: Detection of Syntax Errors (Panic-Mode & Phrase-Level)

Topic 2: Grammar Rewriting (Left Recursion & Left Factoring)

Topic 4: Non-Recursive Top-Down Parsers (Predictive Stack & Buffer)

Topic 6: Bottom-Up Parsers (Shift, Reduce & Handle Pruning)

Topic 8: Handling Parsing Conflicts (Shift-Reduce & Reduce-Reduce)

Topic 10: Reporting of Syntax Errors & Informative Diagnostics

Topic 1: Introduction to Syntax Analysis & CFG Grammars

Syntax Analysis (Parsing) takes the stream of tokens generated by the lexical analyzer and verifies whether the token sequence conforms to the syntactic rules of the language defined by a **Context-Free Grammar (CFG)** $G = (V, T, P, S)$.

Topic 2: Grammar Rewriting Transformations

1. Immediate Left Recursion Elimination:

For grammar production $A \rightarrow A\alpha_1 \mid A\alpha_2 \mid \dots \mid \beta_1 \mid \beta_2$ where no β_i starts with A :

$$\begin{aligned} A &\rightarrow \beta_1 A' \mid \beta_2 A' \mid \dots \\ A' &\rightarrow \alpha_1 A' \mid \alpha_2 A' \mid \dots \mid \epsilon \end{aligned}$$

2. Left Factoring:

Delays the parsing decision when multiple productions share a common prefix α :

$$A \rightarrow \alpha\beta_1 \mid \alpha\beta_2 \implies A \rightarrow \alpha A', \quad A' \rightarrow \beta_1 \mid \beta_2$$

Topic 3 & 4: Top-Down Parsers (Recursive vs. Non-Recursive)

Top-Down Parser Type	Internal Architecture & Mechanism	Key Limitations & Mitigations
Recursive Descent Parser	Executes a set of mutually recursive C/Java functions where each non-terminal corresponds to a procedure.	Cannot directly handle left recursion (causes infinite loop); requires backtracking without predictive lookahead.
Non-Recursive Predictive Parser	Uses an explicit Parser Stack, an Input Buffer with endmarker '\$', and a parsing table $M[A, a]$.	Requires grammar strictly to be $LL(1)$ without conflicts.

Topic 5: Design of LL(1) Parsers (FIRST & FOLLOW Inductive Formulations)

Inductive Rules for $FIRST(\alpha)$: 1. If X is a terminal, $FIRST(X) = \{X\}$. 2. If $X \rightarrow \epsilon$, then $\epsilon \in FIRST(X)$. 3. If $X \rightarrow Y_1 Y_2 \dots Y_k$, add all non- ϵ symbols of $FIRST(Y_1)$ to $FIRST(X)$. If $\epsilon \in FIRST(Y_1)$, add $FIRST(Y_2)$, and so on. If all $Y_i \Rightarrow^* \epsilon$, add $\epsilon \in FIRST(X)$.

Inductive Rules for $FOLLOW(A)$: 1. For start symbol S , place endmarker '\$' into $FOLLOW(S)$. 2. For production $A \rightarrow \alpha B \beta$, add $(FIRST(\beta) \setminus \{\epsilon\})$ into $FOLLOW(B)$. 3. For production $A \rightarrow \alpha B$ or $A \rightarrow \alpha B \beta$ where $\epsilon \in FIRST(\beta)$, add all elements of $FOLLOW(A)$ into $FOLLOW(B)$.

Condition for Grammar to be LL(1)

A grammar G is $LL(1)$ if and only if for every pair of productions $A \rightarrow \alpha \mid \beta$: 1. $FIRST(\alpha) \cap FIRST(\beta) = \emptyset$ 2. If $\epsilon \in FIRST(\beta)$, then $FIRST(\alpha) \cap FOLLOW(A) = \emptyset$.

Topic 6 & 7: Bottom-Up Parsers & LR Variants Hierarchy

Bottom-Up Parsers construct the parse tree starting from the terminal leaves at the bottom and reducing back to the start symbol S at the root (Rightmost Derivation in Reverse):

LR Family	Item Representation	Reduction Decision Rule	State Complexity & Relative Power
1. LR(0)	$[A \rightarrow \alpha \cdot \beta]$	Reduce unconditionally in entire row for all input tokens.	Smallest table; least powerful. Prone to conflicts.
2. SLR(1)	$[A \rightarrow \alpha \cdot \beta]$ (Same as LR(0))	Reduce only on input tokens $a \in FOLLOW(A)$.	Same number of states as LR(0); eliminates many conflicts.
3. CLR(1)	$[A \rightarrow \alpha \cdot \beta, a]$ (With 1 Lookahead)	Reduce only on exact propagated lookahead token a .	Most powerful; massive number of states (thousands).
4. LALR(1)	Merged CLR(1) items with identical cores.	Reduce on merged lookahead sets.	Same state count as SLR(1) with near CLR(1) parsing power. Used in Yacc / Bison.

Topic 8 – 10: Handling Parsing Conflicts & Error Recovery

1. Parsing Conflicts:

Shift-Reduce (S-R) Conflict: The parser cannot decide whether to shift the incoming token or reduce the stack handle (e.g., the classic Dangling Else ambiguity).

Reduce-Reduce (R-R) Conflict: The parser has two distinct valid reduction rules available for the same lookahead token, indicating deep grammar ambiguity.

2. Syntax Error Recovery Techniques:

Panic-Mode Recovery: Parser discards input tokens one by one until a designated *synchronizing token* (e.g., `;` or `}`) is found, then pops non-terminals to resume parsing.

Phrase-Level Recovery: Parser performs local correction on the remaining input (e.g., inserting a missing semicolon or replacing a comma with a semicolon).

Error Productions: Grammar is augmented with explicit rules that anticipate common syntactic mistakes (e.g., `5x` instead of `5 \* x`).

M2 Active Recall & Exam-Important Question Bank

Q1. Given grammar $S \rightarrow (L) \mid a$ and $L \rightarrow L, S \mid S$, eliminate left recursion and compute FIRST and FOLLOW. (8 Marks)

1. Eliminate Left Recursion on L :

$$L \rightarrow SL'$$

$$L' \rightarrow , SL' \mid \epsilon$$

2. Compute FIRST Sets:

$$\text{FIRST}(S) = \{ (, a \}$$

$$\text{FIRST}(L) = \text{FIRST}(S) = \{ (, a \}$$

$$\text{FIRST}(L') = \{ ,, \epsilon \}$$

3. Compute FOLLOW Sets:

$$\text{FOLLOW}(S) = \{ \$,), , \}$$

$$\text{FOLLOW}(L) = \{) \}$$

$$\text{FOLLOW}(L') = \text{FOLLOW}(L) = \{) \}$$